

AIKernel Operator モデルによる依存性最小化並列計算

Prime Phase Generator を事例とした Lean-C-C# 外部化 Operator 設計

Takuya Sogawa

2026-06-02

AIKernel Operator モデルによる依存性最小化並列計算

Prime Phase Generator を事例とした Lean-C-C# 外部化 Operator 設計

Author: Takuya Sogawa

Affiliation: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

Version: v0.1.0

Status: Technical Note / Companion Japanese Translation

Based on: AIKernel.RH v1.5.3

Date: 2026-06-02

DOI: 10.5281/zenodo.20493017

License: CC BY 4.0

Canonical version: English (paper-en.md / paper-en.pdf)

Japanese version: Companion translation (paper-ja.md / paper-ja.pdf)

概要

本稿は、AIKernel における **Operator モデル**を、依存性を最小化した並列計算アーキテクチャとして整理する技術ノートである。事例として、AIKernel.RH において用いられる Prime Phase Generator、すなわち位相干渉モデルに基づく素数生成・評価系を取り上げる。

本稿でいう「依存性最小化」とは、ランタイムシステム全体が依存関係を持たないという意味ではない。各入力単位の意味的評価が、他の入力単位の評価結果や共有可変状態に依存しない、という限定された意味である。この制約の下では、各入力値を独立した Operator 呼び出しとして扱うことができ、Provider 層はチャンク化、バッチ化、スケジューリング、実行資源への配置を自由に選択できる。

AIKernel.RH v1.3.0 から v1.5.3 にかけて、位相干渉計算は、Lean 4 による形式的意味づけ、C によるネイティブ純粋計算、C# による Provider レベルのオーケストレーションという三層構造へ整理された。これにより、数学的に意味づけられた計算コアを、ランタイムのスケジューリング、リソースルーティング、観測、キャンセル、リプレイから分離された Operator として扱えるようになる。

本稿の目的は、既存の素数生成法や素数判定法に対する性能優位性を主張することではない。Prime Phase Generator を具体例として、純粋計算を AIKernel Operator として外部化し、依存性を最小化したチャンク単位・バッチ単位・並列 Provider モデルで実行する一般的な設計を示すことにある。

キーワード

AIKernel; Operator Model; Interface-Led Architecture; Parallel Computing; Lean 4; Native Operator; Prime Phase Generator; Phase Interference; C#; P/Invoke; Lock-Free Computation; TensorKernel

1. 序論

現代の計算環境では、メニーコア CPU、SIMD 命令、GPU、NPU、専用アクセラレータなどにより、ハードウェア側の並列処理能力が急速に拡大している。しかし、ソフトウェアアーキテクチャがこの能力を自動的に活用できるわけではない。共有可変状態、ロック、逐次的なデータ依存、I/O 待機、メモリ帯域、スケジューリングオーバーヘッド、結果集約などは、アプリケーションのスケールを制約する。

AIKernel の **Operator モデル**は、この問題に対してアーキテクチャレベルの整理を与える。Operator は、意味的に閉じた計算単位である。理想的な Operator は、入力を受け取り、決定論的な結果を返し、外部状態を変更せず、他の Operator 呼び出しの結果に依存しない。

問題をこのような独立した Operator 評価の集合として表現できる場合、ランタイムは入力空間を分割し、各分割を並列に評価できる。AIKernel では、これは単なる実装上の工夫ではない。Interface-Led Architecture に基づく契約として、Operator が「何を計算するか」を定義し、Provider が「どこで、どのように実行するか」を決定し、Observer が「何が起きたか」を記録する。

本稿では、Prime Phase Generator を事例として用いる。このモデルでは、自然数 n の干渉エネルギーを評価するために、 $n - 1$ 、 $n + 1$ 、あるいは他の候補値の評価結果を必要としない。この性質により、探索範囲をチャンク化し、各チャンクをバッチとして C# からネイティブ C Operator へ投入する実行モデルが構成できる。

本稿の中心的な問いは、次のように表せる。

形式的に意味づけられた純粋計算を、どのようにネイティブ Operator として外部化し、データ依存性と同期を最小化しつつ、観測可能性を保った Provider で実行できるか。

2. Operator としての位相干渉計算

位相干渉モデルは、素数性を「非自明な内部干渉を持たない状態」として捉える。合成数は内部に約数構造を持ち、素数は $2 \leq n$ という領域条件のもとで非自明な約数を持たない。この直観に基づき、`interferenceEnergy` 関数を考える。

アーキテクチャ上重要なのは、このエネルギー関数の数学的細部そのものではなく、その計算形状である。ある入力の評価は、他の入力の評価から独立している。

概念的には、計算は次のように表せる。

```
InterferenceOperator : Nat -> InterferenceEnergy
```

安定点または素数候補の判定は、次のように表現できる。

```
StablePointOperator : Nat -> Bool
```

Operator 契約には次の性質が求められる。

性質	意味
決定論性	同じ入力と同じ出力を返す。
局所性	計算は入力値のみに依存する。
副作用なし	共有ランタイム状態を変更しない。
再入可能性	複数呼び出しを独立に評価できる。
バッチ可能性	入力列を単位として評価できる。
観測可能性	結果とメトリクスを外部で記録できる。

これは一般に **embarrassingly parallel** と呼ばれる負荷に近い。ただし AIKernel における重要点は、この独立性を意味論的契約へ格上げすることである。これにより Provider は、実行順序、チャンクサイズ、配置先、並列度を変更しても Operator の意味を変えずに済む。

3. AIKernel における Operator / Provider / Observer 分離

AIKernel は計算を三つの責務へ分離する。

役割	責務
Operator	純粋な意味的計算を定義する。
Provider	Operator の実行をランタイム資源へ対応づける。
Observer	結果、メトリクス、リプレイ証跡を収集する。

この分離により、計算そのものがスケジューリング、ログ出力、スレッド管理、リソースルーティングと絡み合うことを避けられる。Operator が純粋であれば、Provider は同じ Operator を .NET managed 実装、ネイティブ C 実装、SIMD 実装、GPU 実装、あるいは将来の TensorKernel backend に配置できる。

Prime Phase Generator では、Operator は各入力値に対して位相干渉エネルギーまたは安定点判定を評価する。Provider は探索空間をチャンクへ分割し、ネイティブ Operator をバッチ単位で呼び出す。Observer はスループット、レイテンシ、キャンセル挙動、結果件数、リプレイ可能な実行証跡を記録する。

この境界は次のように整理できる。

```
Operator: what is computed
Provider: where and how it is executed
Observer: what evidence is preserved
```

この境界を厳密に保つことにより、アーキテクチャは実行環境に対して移植性を持つ。

4. Operator 外部化 : Lean -> C -> C#

AIKernel.RH v1.3.0 から v1.5.3 にかけて、Prime Phase Generator の外部化経路が整理された。ここでいう外部化とは、数学的意味づけ、ネイティブ計算、実行オーケストレーションを明確に分離する設計を指す。

4.1 Lean 4 による形式的意味づけ

Lean 4 は、モデルの数学的意味論を表現するために用いられる。たとえば次の関係が形式的に整理される。

```
Nat.Prime n
interferenceEnergy n = 0
isStableFixedPoint n
```

重要な境界条件として、`interferenceEnergy n = 0` だけでは `0` や `1` も含まれ得る。そのため、自然数全域で素数性と同値な安定固定点として扱うには、`2 <= n` のような領域条件が必要である。

Lean の役割は、スレッドを管理したり実行を最適化したりすることではない。Operator が満たすべき数学的契約を定義することである。

4.2 C によるネイティブ Operator 層

C 層は、純粋計算コアをネイティブ Operator として実装する。この層はマネージドランタイム上の割り当てやオブジェクト管理を避け、バッチ実行に適した狭い関数境界を提供する。

ネイティブ Operator には、次の性質が望ましい。

- ・プリミティブな数値入力または連続バッファを受け取る。
- ・決定論的な出力を返す。
- ・グローバルな可変状態に依存しない。
- ・ランタイム契約上可能であれば並行呼び出しできる。
- ・境界横断コストを下げるためバッチ入力をサポートする。
- ・Provider がチャンクサイズと並列度を制御できる。

この設計では、C は単なる高速化実装ではなく、Operator の外部化された実行形態である。

4.3 C# による Provider オーケストレーション

C# 層は AIKernel.NET 形式の Provider として動作する。計算の数学的意味を再実装するのではなく、実行を制御する。

典型的な Provider の責務は次の通りである。

- 探索範囲をチャンクへ分割する。
- ネイティブ入力バッファを準備する。
- 並列度を制御する。
- バッチ化によって P/Invoke 境界コストを削減する。
- キャンセルを伝播する。
- 結果をストリーミングまたは集約する。
- Observer にメトリクスを送信する。
- リプレイ可能な実行記録を保存する。

この構成により、Lean は意味論を定義し、C は純粋計算を評価し、C# は実行計画を組み立てる、という明確な分離が得られる。

5. 並列化戦略

AIKernel.RH の開発過程では、C# からネイティブ Operator を呼び出すために複数の戦略が検討された。

5.1 SingleCall Parallel

最も単純な戦略は、入力値ごとにネイティブ Operator を呼び出す方式である。

```
for each n in range:
    call native_operator(n)
```

この方式は実装が容易であり、入力単位の独立性を直接示せる。しかし、各入力ごとに P/Invoke 境界を越えるため、入力ごとの計算が軽量な場合には境界コストが支配的になる可能性がある。

そのため、この方式は Operator の意味的独立性を確認する初期実装としては有効だが、大規模探索における最終形としては限定的である。

5.2 Batched Execution

第二の戦略は、複数の入力値をまとめて一度にネイティブ関数へ渡す方式である。

```
call native_operator_batch(values[])
```

バッチ化により、境界横断回数を削減できる。軽量の Operator を大量に評価する場合には特に有効である。一方で、単一の巨大バッチに依存すると、観測性と制御性が低下しやすい。キャンセル応答が遅くなり、進捗報告が粗くなり、メモリ使用量の管理も難しくなる。

5.3 Chunked Batched Parallel

AIKernel.RH v1.5.3 の実用的な到達点は、**Chunked Batched Parallel** 実行である。Provider が探索範囲を論理的チャンクへ分割し、各チャンクをバッチとしてネイティブ Operator に投入する。複数チャンクは並行に評価できる。

```
range -> chunks -> batched native calls -> result streams
```

概念的な実行フローは次の通りである。

```
for each chunk in partition(range):
    run async:
        call native_operator_batch(chunk)
        emit results
```

この方式には次の利点がある。

- SingleCall Parallel より P/Invoke 呼び出し回数を減らせる。

- 単一巨大バッチより観測性が高い。
- チャンク単位でキャンセルと進捗報告ができる。
- 作業単位を独立にスケジュールできる。
- 共有可変状態への依存を減らせる。
- バッチサイズと並列度を調整できる。

ここで重要なのは、チャンク間が意味的に独立していることである。各チャンクの結果は後段で集約すればよく、個別の Operator 評価の意味は変わらない。

6. 評価範囲

本稿 v0.1.0 はアーキテクチャドラフトであり、完全なベンチマークスイートはまだ含まない。今後の版では、少なくとも次の指標を測定する必要がある。

指標	目的
Throughput	1 秒あたりの評価入力数。
Boundary cost	single call と batched call の境界コスト差。
Batch sensitivity	バッチサイズ変更によるスループット変化。
Parallel efficiency	worker 数増加に対するスケール挙動。
Memory pressure	実行時の割り当てとバッファ圧。
Cancellation latency	チャンク実行を停止するまでの時間。
Observer overhead	メトリクスと結果回収のコスト。
Replay overhead	監査可能な証跡保存のコスト。

作業仮説は次の通りである。

入力単位間のデータ依存性を持たない Operator では、
Chunked Batched Parallel 実行は SingleCall Parallel より
境界コストを下げつつ、単一巨大バッチより高い観測性と
制御性を保てる可能性がある。

これはアーキテクチャ上の仮説であり、ベンチマーク結果ではない。今後の実測によって検証される必要がある。

7. 考察

7.1 依存性最小化の意味

「依存性最小化」という表現は精密に解釈する必要がある。システム全体に依存関係がないという意味ではない。Provider は OS、スケジューラ、メモリ、ネイティブライブラリ、ランタイム構成、ハードウェアに依存する。Observer はログ、メトリクス、ストレージに依存する。

本稿の主張はより狭い。

ある入力値の意味的評価が、別の入力値の意味的評価結果に
依存しない。

この性質により、並べ替え、分割、バッチ化、並列スケジューリングの自由度が得られる。

7.2 Amdahl の法則との関係

本アーキテクチャは Amdahl の法則を無効化しない。入力生成、チャンク管理、境界横断、結果集約、メモリ帯域、観測などの逐次的または共有的な要素は残る。

Operator モデルの価値は、これらの逐次部分を純粋計算の外側へ局所化する点にある。Operator / Provider / Observer を分離することで、独立に実行可能な負荷部分を最大化する。

つまり AIKernel は、すべてのランタイム制約を消すのではなく、計算契約から不要な意味的依存を取り除く。

7.3 素数生成を超えた一般性

Prime Phase Generator は事例であり、唯一の対象ではない。同じ構造は、次の条件を満たす負荷に適用できる可能性がある。

1. 各入力を独立に評価できる。
2. 評価が決定論的である。
3. 出力契約が明確である。
4. バッチ化できる。
5. 結果を安全に集約できる。
6. 観測を計算から分離できる。

応用候補として、次の領域が考えられる。

- SAT の候補割当評価。
- 暗号解析における候補鍵探索。
- 素因数分解ワークフローにおける候補除数評価。
- 大規模パラメータ探索。
- 独立サンプル型シミュレーション。
- ルール検証のバッチ評価。
- AI ガバナンスパイプラインの決定論的前処理。

ただし、各領域において独立性が本当に成立するかは個別に確認する必要がある。AIKernel は独立性を仮定するのではなく、Operator 契約として明示する。

8. TensorKernel への展望

AIKernel プロジェクトの長期的な方向性として **TensorKernel** がある。これは、純粋 Operator を CPU、SIMD、GPU、NPU、分散ノード、専用アクセラレータなどヘルレーティングする実行層である。

Operator が安定した意味的契約を持つなら、Provider は計算の意味を変えずに backend を選択できる。

実行資源	適した負荷形状
CPU	分岐の多い整数計算。
SIMD	同型の数値評価の反復。
GPU	大量のデータ並列計算。
NPU	テンソル化された近似評価。
Distributed nodes	大きく分割可能な探索空間。
Quantum accelerator	QUBO などへ写像できる特殊問題。

Prime Phase Generator は、この方向へ向けた初期例である。形式的に意味づけられた計算を純粋 Operator として外部化し、Provider がそれをオーケストレーションする。

9. 非主張事項

過大な解釈を避けるため、本稿は以下を主張しない。

1. 既存の素数生成法や素数判定法に対する性能優位性。
2. すべての計算を依存性のない Operator に分解できること。
3. Amdahl の法則を無効化すること。
4. C 実装が Lean から完全自動抽出され形式的に同一であること。
5. GPU、NPU、量子実装が完了していること。
6. 実測公開前の絶対的ベンチマーク性能。

本稿の主張はアーキテクチャ上のものである。入力単位が意味的に独立した計算について、AIKernel Operator モデルは純粋計算を外部化し、チャンク化・バッチ化・観測可能な Provider によって実行する設計規律を提供する。

10. 結論

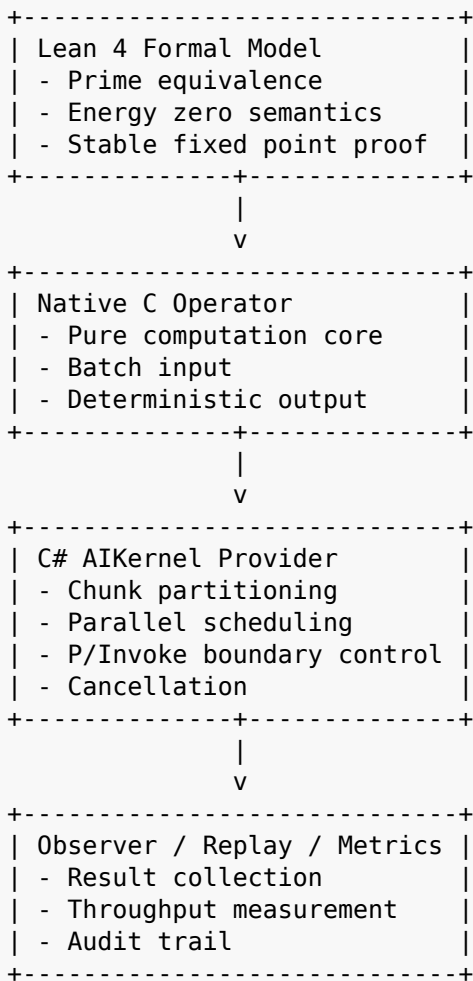
本稿では、AIKernel Operator モデルに基づく依存性最小化並列計算アーキテクチャを提示した。Prime Phase Generator を事例として、形式的に意味づけられた計算を Lean-C-C# 外部化 Operator パイプラインへ整理する方法を示した。

Lean は形式的な意味契約を与える。C はネイティブな純粋計算を提供する。C# は Provider として、チャUNK化、バッチ化、スケジューリング、キャンセル、観測、リプレイを制御する。

このアーキテクチャは、意味と実行を分離する。特定の実装戦略に閉じず、特定のランタイムにも固定されない。純粋計算を Provider 経由で異なる実行資源へ配置しながら、その意味契約を維持できる。

今後は、ベンチマーク測定、バッチサイズ調整、Observer コスト分析、GPU/NPU Provider の探索、TensorKernel 設計との統合を進める必要がある。

Appendix A. Minimal Architecture Sketch



Appendix B. Provider-Level Execution Sketch

```
public async Task<IReadOnlyList<PrimeCandidateResult>>
    EvaluateRangeAsync(
        ulong start,
        ulong end,
        int chunkSize,
        int maxDegreeOfParallelism,
        CancellationToken cancellationToken)
```

```

{
    var chunks = PartitionRange(start, end, chunkSize);

    var options = new ParallelOptions
    {
        MaxDegreeOfParallelism = maxDegreeOfParallelism,
        CancellationToken = cancellationToken
    };

    var results = new ConcurrentBag<PrimeCandidateResult>();

    await Parallel.ForEachAsync(
        chunks,
        options,
        async (chunk, ct) =>
        {
            var input = CreateBatch(chunk);

            var output =
                NativePrimePhaseOperator.EvaluateBatch(input);

            foreach (var item in output)
            {
                results.Add(item);
            }

            await Task.CompletedTask;
        });

    return results.ToArray();
}

```

このコードは概念例である。実装では、ネイティブバッファの所有権、割り当て削減、例外変換、キャンセル伝播、メトリクス送信、リプレイ記録、結果順序などをより厳密に扱う必要がある。

References

- Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. AFIPS Conference Proceedings, 30, 483-485. <https://doi.org/10.1145/1465482.1465560>
- Gustafson, J. L. (1988). Reevaluating Amdahl's Law. Communications of the ACM, 31(5), 532-533. <https://doi.org/10.1145/42411.42415>
- de Moura, L., Kong, S., Avigad, J., van Doorn, F., & von Raumer, J. (2015). The Lean Theorem Prover. Automated Deduction - CADE-25, Lecture Notes in Computer Science, 9195, 378-388. Springer. https://doi.org/10.1007/978-3-319-21401-6_26
- The mathlib Community. (2020). The Lean mathematical library. Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, 367-381. <https://doi.org/10.1145/3372885.3373824>
- Sogawa, T. (2026). Phase-Interference Energy and the Formal Structure of the PG1224 Prime Generation System: A Lean 4 Formalization of Prime = Energy 0 = Stable Fixed Point. Zenodo. <https://doi.org/10.5281/zenodo.20483437>
- Sogawa, T. (2026). AIKernel Trajectory Governance Model: A Kernel-Level Framework for Convergent Decision Control over Stochastic Language Model Inference. Zenodo. <https://doi.org/10.5281/zenodo.20290614>
- Sogawa, T. (2026). AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems. Zenodo. <https://doi.org/10.5281/zenodo.20460322>